Interview prep

This is the one doc to study from. Read it top to bottom, then use the question bank at the end to quiz yourself: cover the answer, say yours out loud, compare. The two things to be smoothest on are the prefill-vs-decode bottleneck split and the cross-check story, because those are what make you sound like you actually did the work. TEACHING.md explains anything here from scratch.

The 60-second pitch

"I built a single-GPU LLM inference benchmarking harness for vLLM. It serves a model behind an OpenAI-compatible API, drives it with a load generator I wrote, watches the GPU with nvidia-smi, and finds the latency-throughput knee. The part I'm proudest of: I cross-checked my numbers against vLLM's official benchmark, which caught a real bug in my own load generator, and once I fixed it everything lined up to within a few percent."

The result (single A100 40 GB, Llama-3.1-8B bf16, 512-in / 128-out)

- Raw throughput ceiling about 16 req/s, roughly 2,000 output tokens/sec.
- Useful capacity under an SLO (TTFT under 1 s and TPOT under 50 ms) peaks around 7 req/s; goodput collapses to zero by rate 32.
- Bottleneck is memory-bandwidth-bound decode: GPU util ~89%, but power plateaued around 350 W (below the card's limit) and KV-cache topped out at ~76% (never full). TPOT is the metric that crosses the SLO first; TTFT stays low until very high load.
- Cross-validated against vLLM's official `vllm bench serve` to within about 1-6%.

How it maps to the resume

1. *Dockerized single-GPU vLLM harness, OpenAI API* — vLLM serves Llama-3.1-8B behind `/v1/completions`; one GPU on purpose (isolate variables); Docker for a reproducible, portable run.
2. *Sweeps over rate / concurrency / prompt / output, measuring TTFT, TPOT/ITL, throughput, P95/P99, failures* — the orchestrator runs the grid; metrics are computed once, the right way (window-based throughput, percentiles with a min-sample guard, failures excluded from latency).
3. *nvidia-smi GPU telemetry correlated with serving* — a background sampler logs util, HBM, power against each load level; that's what produced the bottleneck diagnosis.
4. *CSV/JSON reports + plots for tradeoffs and saturation* — raw JSONL per request, one-row-per-cell summary, and four charts including the knee.

Deep dives — understand these cold

Prefill vs decode (the two phases)

Every request has two phases. **Prefill** runs the whole prompt through the model in one forward pass, fills the KV cache for those tokens, and produces the first output token. It's a big matrix-matrix multiply, so it's compute-bound, and it sets **TTFT** (time to first token). **Decode** then generates one token per forward pass, each attending to all the cached KV, and it sets **TPOT** (time per output token). The reason I measure them separately rather than just end-to-end latency is that they bottleneck on different hardware, so they tell you to fix different things.

Why decode is memory-bandwidth bound

It comes down to arithmetic intensity. To generate one token in decode, the model streams all ~16 GB of bf16 weights out of memory and does only about 1 FLOP per byte with them. The math units finish and then sit idle waiting for the next weights to arrive, so the limit is how fast you can move weights off HBM, not how fast you can multiply. My telemetry confirmed it: util read ~89% (looks busy) but power held around 350 W, below the card's limit. If it were genuinely compute-saturated, power would pin near the limit because the tensor cores would be fully lit. Power low while util high is the fingerprint of stalling on memory. And behaviorally, TPOT (the decode metric) crossed the SLO first while TTFT (the prefill metric) stayed fine, which is the same story from the other side.

Continuous batching, PagedAttention, and the KV cache

vLLM does **continuous batching**: instead of waiting for a fixed batch to finish, it adds and removes requests from the running batch every decode step, so the GPU never idles on stragglers. The reason this raises throughput ties straight to the bandwidth point: every forward pass pays the ~16 GB weight read regardless of how many requests are in the batch, because they all share that one read. One request per pass means 16 GB for one token; 32 requests batched means 32 tokens for the same 16 GB. So batching amortizes the expensive memory traffic. The cost is per-request latency, since a request now shares each pass and waits behind the others, which is why TPOT rises as the batch grows. That tension is exactly what the knee chart maps.

PagedAttention is what makes that practical. The KV cache grows as a request generates and you don't know the final length, so naive systems over-reserve a contiguous block per request and waste 60-80% of KV memory. PagedAttention stores the KV in fixed-size pages that don't have to be contiguous, with a per-request block table, so it allocates on demand and wastes almost nothing. Less waste means more concurrent requests fit, which means bigger batches, which is what gives you the throughput.

The **KV cache** itself is the per-request Key/Value vectors kept in GPU memory so attention doesn't recompute them every step. For Llama-3.1-8B it's about 128 KiB per token. The formula is 2 (K and V) × layers × KV-heads × head-dim × bytes, and the classic trap is using 32 for the head count. Llama-3.1-8B uses Grouped-Query Attention, so the KV cache is sized by `num_key_value_heads` = 8, not the 32 query heads. Using 32 overstates it by 4x.

One clarification on "KV-cache topped out at 76%": that's 76% of vLLM's KV *budget*, not 76% of the 40 GB card. On a 40 GB A100 with gpu-memory-utilization 0.9, vLLM works inside ~36 GB; weights take ~16 GB, leaving roughly 17-18 GB for KV. So 76% means about 13 GB of KV in use. The point that matters is it never hit 100%, so capacity wasn't the wall.

Memory bandwidth vs VRAM capacity (two different specs)

People lump these together but they're separate. **VRAM capacity** (GB) decides whether the model fits and how many requests you can batch. **Memory bandwidth** (GB/s) decides how fast tokens come out. A card can have lots of one and little of the other. So "is 48 GB worse than 128 GB" depends on which you mean: more GB lets you fit bigger models and batch more (throughput), but it does not by itself make tokens faster, that's bandwidth. My bottleneck was bandwidth, and I only used 76% of the KV budget, so I had capacity to spare; more VRAM would have let me batch more but wouldn't have sped up each token.

What decides bandwidth (HBM vs GDDR)

Mostly the memory type. Consumer cards use GDDR; datacenter cards use HBM, which is stacked next to the die with a hugely wide bus, giving multi-TB/s versus under 1 TB/s for GDDR. Bandwidth is roughly bus-width times data-rate, and HBM wins by having a very wide bus. Rough numbers: A10G ~600 GB/s, L40S ~860 GB/s, RTX 4090 ~1 TB/s, A100 ~1.5-2 TB/s, H100 (HBM3) ~3.3 TB/s, H200 (HBM3e) ~4.8 TB/s. For serving, you check the GB/s and whether it says HBM, not just the GB.

Precision as a bandwidth lever (bf16 / fp8 / int4)

The precision is literally the "bytes per parameter" in that 16 GB-per-token number. I ran bf16, which is 2 bytes (16 GB). fp8 is 1 byte, so you move half the data per token, roughly 2x faster decode. int4 is half a byte, roughly 4x. It's not free, though: lower precision costs some quality (int4 more than fp8), and fp8 needs hardware support (H100 or Ada). So quantization attacks the exact bottleneck I measured, because decode is bandwidth-bound and you're shrinking the data moved.

The H100 prediction (what the bandwidth ratio does and doesn't predict)

Since the bottleneck is bandwidth, the bandwidth-bound metrics should improve roughly with the bandwidth ratio. My A100 was ~1.5-1.6 TB/s, an H100 is ~3.3 TB/s, so about 2x, which predicts roughly 2x on TPOT, output tok/s, and goodput. What it does *not* predict is a blanket 2x on everything, TTFT comes from prefill, which is compute-bound and improves on its own schedule, and in practice an H100 often beats the raw ratio because of more compute, bigger cache, and fp8 support. The honest way to turn the prediction into a fact is to run the same benchmark on an H100, which the harness is built to do.

The cross-check bug (the story to lead with)

I built my load generator, got numbers that looked plausible, TTFT "exploding" to ~2.4 s under load. Then I pointed vLLM's official tool at the same server. Throughput, TPOT, and end-to-end latency matched within ~10%, but my TTFT was 4.7x higher than the oracle's (746 ms vs 159 ms). To find out why, I split service-time TTFT from a coordinated-omission-corrected one and looked at schedule_delay, the gap between when I intended to send and when I actually sent. It was 1.5 ms, so I'd sent on time, which ruled out coordinated omission. The real cause was my httpx connection pool (sized 80) being smaller than the peak in-flight requests (141), so ~60 requests queued inside my own client before reaching vLLM, inflating TTFT (and deflating TPOT because tokens then arrived in bursts). I sized the pool to the peak and everything converged to the oracle. The bug had also pointed me at the wrong diagnosis (blame the queue); the corrected data pointed at decode/bandwidth, which matched the GPU telemetry. That's the whole reason you cross-check, a self-built ruler can be confidently wrong.

Reading the four graphs

- **Latency-throughput knee** (output tok/s vs P99 E2E): each dot is a load level. Both rise with load, but throughput hits a ceiling (bandwidth) while latency doesn't (queue grows), so the curve bends up sharply at the knee. Run just left of it.
- **Goodput vs raw throughput**: raw is everything completed; goodput is only what met the SLO. They track until ~7 req/s, then goodput falls while raw keeps rising, the gap is work too slow to be useful.
- **TTFT and TPOT vs load**: splits the latency so you see TPOT cross the SLO first (decode is the weak link) while TTFT holds until very high load.
- **GPU saturation**: util/power/KV vs load plus a time series. Util high, power below limit, KV not full, that combination is the memory-bandwidth diagnosis.

Why the numbers are trustworthy (and the honest limits)

Three things: the cross-check against vLLM's official tool (using the same gamma arrival law so it's apples-to-apples), pinned versions and seeds plus an environment manifest in every run, and raw per-request JSONL so any metric can be recomputed by a single aggregation path. One honest caveat worth stating: the cross-check lines up on the service-time-comparable metrics; my headline TTFT is the coordinated-omission-corrected version, which under saturation can read a bit higher than the oracle's service-time number, and that's deliberate, it's the more honest, user-felt latency. And I developed the whole thing on a Mac with no GPU by running everything against a mock vLLM server, which is how the metric and plotting code is unit-tested offline.

Question bank

Generated from six interviewer angles (fundamentals, hardware, measurement, debugging, systems/scaling, behavioral) plus a gap pass. Cover the answer, say yours, compare.

Fundamentals: inference internals

Walk me through the two phases of an LLM inference request. What's the difference between prefill and decode, and which latency metric does each one drive?

warmup · Prefill vs decode

An inference request has two distinct phases. Prefill processes the entire prompt in a single forward pass — all the prompt tokens go through the model at once, the attention is computed over the whole sequence, and the KV cache for those tokens gets populated. That one pass produces the first output token. Because you're crunching the whole prompt in parallel, prefill is compute-bound — it's a big matmul that keeps the tensor cores busy. Prefill is what sets TTFT, time-to-first-token, since the user waits for that whole pass before seeing anything. Then decode takes over: it's autoregressive, generating one token per forward pass, each new token attending to all the cached KV from before. Decode is what sets TPOT, time-per-output-token, the inter-token latency once streaming starts. So in my harness TTFT is dominated by prefill plus any queueing, and TPOT is pure steady-state decode. The important consequence is they bottleneck on different hardware resources, which is why I measure them separately rather than just reporting end-to-end latency.

> Follow-ups: If a request has a 512-token prompt and generates 128 tokens, roughly how many forward passes happen in each phase? · Why can't you just report a single end-to-end latency number and call it done? · How does prompt length affect TTFT versus TPOT?

You claim decode is memory-bandwidth-bound rather than compute-bound. Defend that with the arithmetic, and tell me what telemetry in your run actually confirmed it.

core · Why decode is memory-bandwidth bound

The core argument is arithmetic intensity. In decode you generate one token at a time, so each forward pass for a single request does a matrix-vector product against the weights, not a matrix-matrix. For an 8B model in bf16 that's about 16GB of weights, and every single token you generate has to stream all ~16GB out of HBM. But the amount of compute per byte read is tiny — on the order of 1 FLOP per byte — so the tensor cores finish their math and then sit idle waiting for the next chunk of weights to arrive from memory. You're memory-bandwidth-bound: the bottleneck is how fast you can move weights off HBM, not how fast you can multiply. My telemetry confirmed exactly this. On the A100 GPU utilization read ~89%, which looks busy, but power plateaued around 350W, well below the card's TDP — if it were genuinely compute-saturated the power would pin near the limit because the math units would be fully lit. Power staying low while util is high is the signature of stalling on memory. And TPOT, the decode metric, is what crossed my SLO first; TTFT, the compute-bound prefill metric, stayed low until very high load. That pattern — decode degrades before prefill — is the behavioral fingerprint of a memory-bandwidth wall.

> Follow-ups: Why does GPU utilization read 89% if the compute units are stalling — isn't util supposed to mean 'doing work'? · If decode is memory-bound, what's the single most effective hardware change to speed it up? · How does batching change the arithmetic intensity of decode, and why does that help?

Explain continuous batching as vLLM implements it, and why it raises throughput. What's the cost it imposes?

core · Continuous batching

Naive static batching waits to assemble a fixed batch, runs all of them to completion together, and the whole batch is held hostage by its longest-generating member — short requests finish and their slots sit idle while the long one keeps going. Continuous batching, sometimes called iteration-level or in-flight batching, operates at the granularity of a single decode step instead. After every token-generating iteration, the scheduler can evict requests that just finished and slot newly-arrived requests into the running batch, including running their prefill. So the batch composition changes token-by-token and the GPU never idles waiting for stragglers. The reason it raises throughput connects directly to the memory-bandwidth story: in decode you pay ~16GB of weight reads per forward pass regardless of how many requests are in the batch, because all requests in a batch share that single weight read. So if you have one request per pass you've 'spent' 16GB to produce one token; with 32 requests batched you produce 32 tokens for the same 16GB read. You're amortizing the expensive memory traffic across the batch. The cost is per-request latency: a request now shares each forward pass and waits behind the others in the batch, so TPOT goes up as the batch grows. That's the throughput-versus-latency tension my whole knee analysis is mapping — batching buys aggregate tokens/sec at the price of individual TPOT, and the SLO is where that trade stops being worth it.

> Follow-ups: If a new request arrives mid-batch, does it have to wait for the current requests to finish before its prefill runs? · How does the scheduler decide how many requests to admit into the running batch at once? · Why does throughput plateau around 16 req/s rather than scaling indefinitely with batch size?

What problem does PagedAttention solve, and how does it relate to continuous batching working well in practice?

core · PagedAttention

PagedAttention solves KV-cache memory fragmentation. The KV cache for a request grows as it generates tokens, and you don't know the final length in advance. The naive approach reserves a contiguous block of GPU memory sized for the max possible sequence length per request, which wastes huge amounts of memory — internal fragmentation from over-reservation, plus external fragmentation as requests of different sizes come and go. PagedAttention borrows the idea of virtual memory paging from operating systems: it splits the KV cache into fixed-size blocks, or pages, that don't have to be physically contiguous. Each request keeps a block table mapping its logical token positions to wherever those pages physically live in GPU memory, and the attention kernel is written to gather KV across those scattered pages. The payoff is you allocate KV memory on demand, page by page, so there's almost no waste — vLLM reports near-zero fragmentation versus 60-80% waste in naive systems. That's exactly what makes continuous batching practical: because you're not over-reserving per request, you can pack far more concurrent requests into the same VRAM, which means a bigger running batch, which is what amortizes the weight reads and gives you the throughput. It also enables cheap sharing — prefix caching and copy-on-write for things like beam search — because shared prefixes can point at the same physical pages.

> Follow-ups: What determines the page/block size, and what's the tradeoff in making it larger or smaller? · How does the block table get used inside the attention kernel at runtime? · When KV memory finally fills up, what does vLLM do — does it drop requests, queue them, or something else?

Size the KV cache for Llama-3.1-8B. Give me bytes per token and walk the calculation — and I want to see you avoid the most common mistake people make here.

hard · KV cache sizing and the GQA trap

The footprint per token comes out to about 128 KiB for Llama-3.1-8B in bf16. The formula is: 2 (one K and one V) times number of layers times number of KV heads times head dimension times bytes per element. The trap — and this is the mistake almost everyone makes — is using 32 for the head count because Llama-3.1-8B has 32 attention query heads. But it uses Grouped-Query Attention, and the KV cache is sized by num_key_value_heads, which is 8, not 32. The query heads are grouped so that every 4 query heads share a single KV head, so you store 8 KV heads, not 32. Using 32 would overestimate the KV cache by 4x — that's the '4x trap.' Plugging it in: 2 K/V * 32 layers * 8 KV heads * 128 head_dim * 2 bytes = 131,072 bytes = 128 KiB per token. GQA is specifically a design choice to shrink the KV cache, because KV cache size is what limits how many requests and how long a context you can fit, and a smaller KV footprint means more concurrency and bigger batches. This matters operationally because KV capacity is the wall: when the cache fills, vLLM has to queue or preempt requests. In my run KV utilization only reached about 76% — it never actually filled — which told me the capacity wall was not my bottleneck; the memory-bandwidth wall on decode hit first.

> Follow-ups: If KV only reached 76%, why didn't throughput keep climbing — what stopped it before the cache filled? · For a fixed VRAM budget, how does GQA change the maximum concurrency you can support versus full multi-head attention? · How would switching weights to fp8 change both the KV math and the decode bandwidth story?

Your SLO is TTFT <= 1000ms AND TPOT <= 50ms. Under increasing load, which one breaks first in your data, why that one, and what does that tell you about the bottleneck?

core · TTFT vs TPOT and the SLO

TPOT breaks first — it crosses 50ms at around request rate 16, while TTFT stays comfortably low until much higher load. That ordering is the whole diagnosis. TTFT is gated by prefill, which is compute-bound and runs as a single efficient parallel pass; under continuous batching, prefill for a new request gets interleaved quickly, so first-token latency stays low even as load climbs. TPOT is gated by decode, which is memory-bandwidth-bound, and as the running batch grows, each decode step has more requests sharing it, so the per-token interval stretches. Since decode is already the memory-bound bottleneck, it's the metric that degrades earliest. So the fact that the per-token metric blows the SLO before the first-token metric is a direct signal that the system is decode-limited, not prefill-limited and not queue-limited — which lined up with the GPU telemetry showing the memory-bandwidth signature. This is also why I refuse to collapse to one latency number: a single E2E figure would have hidden that it's specifically the decode phase failing, and the operational fix for a decode bottleneck (more bandwidth, fp8, smaller batch) is completely different from the fix for a prefill or queueing bottleneck.

> Follow-ups: Why set two separate thresholds instead of one end-to-end latency target — give me a concrete user-facing reason TPOT deserves its own SLO. · If you needed to push the TPOT-50ms crossing point to a higher request rate on the same A100, what would you try first? · How does output length interact with TPOT versus TTFT in the SLO?

Your raw throughput ceiling is ~16 req/s but useful capacity peaks at ~7 req/s and goodput hits zero by rate 32. Reconcile those numbers and tell me what you'd actually report to someone capacity-planning a deployment.

hard · Goodput vs raw throughput

They're measuring two different things. Raw throughput, ~16 req/s or ~2,000 output tokens/sec, is the rate at which the server completes requests regardless of how slow each one was — it's the saturation ceiling of the hardware. Goodput is throughput filtered through the SLO: only requests that met TTFT <= 1000ms and TPOT <= 50ms count. As I push past ~7 req/s, the batch grows, TPOT inflates, and more and more completed requests violate the 50ms TPOT bound — so they still complete and count toward raw throughput, but they stop counting toward goodput. By rate 32 essentially every request is over the SLO, so the server is busy doing ~16 req/s of work that's all useless under the SLO, and goodput collapses to zero. The collapse is worse than a plateau: offering more load actively destroys useful capacity because it bloats the batch and drags everyone over the latency line. What I'd report for capacity planning is the goodput number, ~7 req/s per A100, not the 16. Provisioning against the raw ceiling is how you build a system that looks 'up' on a throughput dashboard while every user is getting unacceptably slow tokens. The actionable headline is: one A100 serves about 7 SLO-compliant req/s for this model and SLO, and you scale horizontally past that.

> Follow-ups: Goodput hitting zero by rate 32 implies overload behaves badly — what admission-control or load-shedding strategy would protect goodput? · Why use P95/P99 percentiles rather than averages when deciding whether a request met the SLO? · How sensitive is that 7 req/s number to the SLO thresholds — if I relax TPOT to 75ms, roughly what happens?

Tell me about a time your benchmark lied to you. How did you catch it, and how did you rule out the obvious culprit before finding the real one?

hard · Measurement methodology / the load-gen bug

An early run reported TTFT about 4.7x too high — 746ms versus the official 'vllm bench serve' oracle's 159ms at the same point. The obvious culprit for inflated tail latency in a load generator is coordinated omission: if your sender slows its own sending under load, it never measures the requests it should have sent during the slow period, and paradoxically it can also distort the latencies it does record. So the first

thing I checked was schedule_delay — the gap between each request's intended absolute send time and when it actually went out, since I pre-schedule send times from a Poisson process and record intended-versus-actual precisely to detect this. schedule_delay was only ~1.5ms, so the sender was firing on time; it was not coordinated omission. The real root cause was the httpx connection pool. I'd sized the pool at 80 connections, but at peak there were 141 requests in flight. So 61 requests had no connection and queued client-side, inside my load generator, before they ever hit the server. That client-side queue inflated TTFT — the clock starts at intended send but the request sat waiting for a connection — and it deflated TPOT because fewer requests were actually concurrent on the server at any instant. The fix was sizing the connection pool above peak in-flight; after that every metric — TTFT, TPOT, E2E — converged to the oracle within about 1-6%. The deeper lesson is the bug didn't just add noise, it pointed at the wrong diagnosis: the bad data looked like server-side queueing, when the corrected data pointed cleanly at decode and memory bandwidth, matching the GPU telemetry. A benchmark that isn't validated against an independent oracle will confidently mislead you.

> Follow-ups: Why does an undersized client connection pool inflate TTFT but deflate TPOT specifically — walk me through the mechanism on both metrics. · How exactly does cross-checking against 'vllm bench serve' give you confidence — couldn't both tools share the same blind spot? · What's your minimum-sample-size guard for, and how does it interact with reporting P99?

Suppose I hand you an H100 instead of the A100. Predict what happens to your numbers and justify it from first principles — and tell me where capacity and bandwidth could mislead each other.

curveball · Hardware extrapolation

My prediction is roughly a doubling of decode throughput, and the harness is the tool I'd use to confirm it rather than just assert it. The reasoning is straight from the bottleneck: decode is memory-bandwidth-bound, so to first order decode speed scales with HBM bandwidth. The A100 sits around 1.5-2 TB/s; the H100 with HBM3 is around 3.3 TB/s — roughly 2x — so streaming the ~16GB of weights per token happens about twice as fast, and tokens come out about twice as fast. An H200 at ~4.8 TB/s would push further. The place these specs mislead each other is conflating VRAM capacity with bandwidth. Capacity, measured in GB, answers 'does the model fit and how many concurrent requests / how much KV can I hold' — it sets the concurrency and batch-size ceiling. Bandwidth, measured in GB/s, answers 'how fast do tokens come out' — it sets decode speed. They're orthogonal. You can have a card with plenty of capacity to fit a huge batch but mediocre bandwidth, so it can hold the requests but decodes them slowly; or vice versa. A consumer GDDR card might have big VRAM numbers but under 1 TB/s, so it fits the model but decodes far slower than an HBM datacenter card. The other lever besides the card is precision: fp8 weights are 1 byte/param instead of bf16's 2, so you move half the data per token and decode roughly doubles on the same hardware; int4 is ~4x. So 'H100' and 'fp8' both attack the same bandwidth bottleneck from different angles, and I'd run the same sweep on each and read the goodput-versus-load curve to verify, since real speedups are always under the SLO, never just the raw ceiling.

> Follow-ups: If decode scales with bandwidth, what happens to prefill / TTFT on the H100 — does it scale the same way? · fp8 doubles decode bandwidth-wise, but what could you lose, and how would you check it's safe? · On the H100 you can now fit a bigger batch — does that help goodput, or just raw throughput? Be careful.

Hardware: GPU, memory, precision

Your writeup is emphatic that VRAM capacity and memory bandwidth are two different specs. Spell out the distinction, and tell me which one your benchmark found to be the bottleneck.

warmup · Memory bandwidth vs VRAM capacity

They answer two completely different questions. VRAM capacity is gigabytes: does the model fit, and how much KV cache and concurrency can you hold at once. Bandwidth is gigabytes per second: how fast you can stream those bytes off HBM, which is how fast tokens come out during decode. You can have plenty of one and run out of the other. In my run on an A100 40GB the model fit fine in bf16 — about 16GB of weights — and KV cache only ever reached about 76 percent, so I was never capacity-bound. The wall was bandwidth: decode is memory-bandwidth-bound, every output token forces a full re-read of the weights from HBM, and that's the spec that capped my throughput. The telemetry confirmed it — GPU util sat around 89 percent but power plateaued near 350W, well under the card's limit, which is the classic signature of compute units stalling on memory rather than doing math.

> Follow-ups: If KV cache had hit 100 percent instead of 76, would that change your bottleneck diagnosis? · Which of the two specs would you over-provision if you could only pick one for an 8B decode-heavy workload?

Walk me through the physics of why decode is memory-bandwidth-bound while prefill isn't. Use numbers.

core · Why decode is bandwidth-bound

It comes down to arithmetic intensity — FLOPs per byte moved. In decode you generate one token per forward pass, so for that single token you have to stream all ~16GB of bf16 weights out of HBM, and you do only on the order of 1 FLOP per byte read. That's a tiny arithmetic intensity, far below the GPU's roofline ridge point, so the tensor cores sit idle waiting on memory — they finish their multiply long before the next chunk of weights arrives. That's why power stays below TDP: you're not doing enough math to draw full power. Prefill is the opposite: you process the entire prompt — 512 tokens in my case — in one forward pass, so you read the weights once but do hundreds of tokens' worth of matmul against them. High arithmetic intensity, compute-bound, sets TTFT. So prefill burns FLOPs and sets time-to-first-token; decode burns bandwidth and sets time-per-output-token. My data matched that exactly — TPOT crossed the 50ms SLO first, at around rate 16, while TTFT stayed low until very high load.

> Follow-ups: Where is the roofline ridge point for an A100 roughly, and why does decode land to the left of it? · Batching raises arithmetic intensity for decode — how, and why doesn't it make decode compute-bound?

You contrast HBM with GDDR. What physically decides bandwidth, and why does a datacenter card hit multiple TB/s while a consumer card stays under 1 TB/s?

core · HBM vs GDDR

Bandwidth is bus width times effective clock — bytes per transfer times transfers per second. HBM wins on width. It's stacked DRAM dies sitting on a silicon interposer right next to the GPU, connected by a very wide bus — thousands of bits — so even at a modest per-pin clock the aggregate is huge. GDDR is discrete chips on the PCB around the die, wired over a much narrower bus, typically a few hundred bits; it compensates with very high per-pin clocks but can't close the gap, so it tops out under 1 TB/s. That's the whole reason A100/H100/H200 are HBM parts and gaming cards are GDDR. The width-over-clock tradeoff also means HBM moves more bytes per watt, which matters when you're bandwidth-bound and already power-limited on the math. Concretely: A100 is roughly 1.5 to 2 TB/s, H100 with HBM3 around 3.3

TB/s, H200 around 4.8 TB/s — and since decode speed tracks bandwidth almost linearly, those numbers are basically a decode-throughput ranking.

> Follow-ups: Two cards with identical TFLOPs, one HBM one GDDR — which serves LLM decode faster and why? · What does the interposer buy you besides width — anything on latency or energy per bit?

You predicted an H100 would roughly double throughput over the A100. Defend that number — what's the reasoning, and what could make it wrong?

hard · The H100 prediction

The logic is: I proved on the A100 that I'm decode-bound, and decode throughput scales with memory bandwidth. H100 HBM3 is about 3.3 TB/s versus the A100's ~1.5-2 TB/s — call it roughly 2x the bytes per second — so to first order I expect about 2x the output tokens per second, because I'm just reading those 16GB of weights faster per token. That's a bandwidth-ratio argument, not a FLOPs argument, which is the right lens for a bandwidth-bound workload. What could make it wrong: if a different bottleneck moves to the front. The H100 also has more FLOPs and more VRAM, so I might be able to run bigger batches before KV cache fills, which could push me past 2x — or some non-bandwidth cost like scheduling, Python overhead in the load path, or kernel launch could become the limiter and I'd get less than 2x. And there's fp8 on Hopper, which is a separate multiplier on top. The honest answer is the harness is the instrument: I'd run the identical open-loop sweep on the H100 and check the throughput ceiling, TPOT-vs-load, and the power/util/KV telemetry to confirm it's still bandwidth-bound at the new ceiling, and cross-check against vllm bench serve.

> Follow-ups: If the H100 came back at only 1.4x, what's the first telemetry you'd look at to explain the shortfall? · Would you expect TTFT to improve by the same ratio as throughput? Why or why not?

Explain how going from bf16 to fp8 to int4 interacts with your bottleneck. Why is precision a bandwidth lever specifically?

hard · Precision and the bandwidth bottleneck

Because the bottleneck is bytes moved per token, and precision is literally how many bytes each weight is. bf16 is 2 bytes per parameter, so 8B params is ~16GB streamed every decode step. fp8 is 1 byte — half the data moved per token — so decode roughly doubles, since I'm bandwidth-bound and I just halved the thing the bound is on. int4 is half a byte, roughly 4x. It's not that the math got faster; it's that the bottleneck resource shrank. There's a second win: lower precision also shrinks the KV cache and the weights' footprint, so I free VRAM capacity, which lets me batch more before hitting the capacity wall — and bigger batches amortize the weight read further. The caveat is that quantization isn't free — there's accuracy degradation and dequant overhead, and the speedup is only that clean because I'm bandwidth-bound; on a compute-bound phase like prefill the win is smaller. fp8 specifically needs hardware support, which is why it pairs naturally with the H100 discussion.

> Follow-ups: If you were compute-bound instead of bandwidth-bound, would fp8 still roughly double decode? Why not? · How would you measure the accuracy cost of int4 so you're not just trading correctness for tokens/sec?

You flag a '4x trap' in KV-cache math for Llama-3.1-8B. What's the trap, what's the right per-token number, and why does it matter for hardware sizing?

hard · KV cache sizing — the 4x trap

The trap is computing KV cache off the 32 attention heads. Llama-3.1-8B uses grouped-query attention with num_key_value_heads = 8, not 32, so the K and V projections you actually cache are 4x smaller than

the naive count. Using 32 heads overestimates KV memory by 4x. The right figure is about 128 KiB per token. Why it matters for hardware: KV cache is the capacity dimension — it grows with sequence length times concurrency, and when it fills, requests queue and you hit the capacity wall. If I size VRAM or pick a card off a 4x-inflated number I'd massively over-provision capacity, or wrongly conclude I'm capacity-bound when I'm not. In my A100 run the correct number is part of why KV only reached 76 percent — I had capacity headroom, which is exactly what let me see that bandwidth, not capacity, was the real wall. Getting this wrong would have pointed me at the wrong bottleneck and the wrong GPU.

> Follow-ups: How does that 128 KiB/token scale if I serve 100 concurrent users at 4K context — does an A100 40GB hold it? · GQA shrinks the KV cache — does it also help the bandwidth bottleneck during decode, or only capacity?

A team asks you to pick a GPU to serve Llama-3.1-8B in production. Given what you measured, how do you reason about the choice?

core · GPU selection for serving

I start by classifying the workload, because the spec that matters depends on it, and my benchmark already told me 8B serving is decode/bandwidth-bound. So my primary lever is memory bandwidth, not peak FLOPs — that's what sets tokens per second under load. Second, I check capacity: the model has to fit plus enough KV cache for my target concurrency and context length, using the correct GQA-based 128 KiB/token, not the 4x-inflated number. For an 8B model that's modest, so capacity rarely forces my hand — bandwidth does. Third, I size against an SLO, not raw throughput: my harness showed raw ceiling around 16 req/s but useful goodput under a TTFT<=1000ms / TPOT<=50ms SLO peaked near 7 req/s, and goodput collapsed past that. So I provision for the goodput knee, not the ceiling. Concretely: A100 if the budget's tight and 7-ish req/s per card meets demand; H100 if I need roughly double the tokens/sec or want fp8; H200 if bandwidth is everything. And I wouldn't guess — I'd run the same open-loop sweep on each candidate and pick on measured goodput-per-dollar.

> Follow-ups: Where does fitting two 8B replicas on one big card beat one model on two smaller cards? · Your SLO knee is 7 req/s but the ceiling is 16 — how does that gap change which card is 'enough'?

Tie this to money. How would you compute cost-per-token across A100 vs H100, and why can the more expensive card be cheaper per token?

hard · Cost per token / $-per-token

Cost per token is dollars-per-hour for the instance divided by tokens-per-second times 3600 — but the tokens/sec has to be goodput at my SLO, not the raw ceiling, or I'm pricing tokens I can't actually sell. An H100 might be roughly 2x the hourly cost of an A100, but if its ~2x bandwidth gives me ~2x the decode throughput, the cost per token is a wash on bandwidth alone — and then it tips in the H100's favor once you add fp8, which can roughly double throughput again for under 2x the bytes, plus the bigger card lets me batch more, amortizing the weight read across more requests and pushing utilization up. So the lever is throughput-per-dollar, and because decode is bandwidth-bound, the card with the best bandwidth-and-precision-per-dollar usually wins per token even if its sticker price is higher. The trap is comparing sticker prices, or dividing by raw ceiling throughput instead of SLO goodput — past the knee my goodput collapses to zero, so those extra 'ceiling' tokens are worthless and would make a card look artificially cheap. I'd settle it empirically: run the open-loop sweep on each, take goodput at the SLO, and divide real instance price by that.

> Follow-ups: Batching lowers cost per token but raises per-request latency — how do you find the batch size that minimizes $/token without breaking the SLO? · How would idle time and bursty traffic change the real

> cost-per-token versus your steady-state sweep number?

Your early run blamed queueing and showed TTFT 4.7x too high. How do you know the real bottleneck is hardware memory bandwidth and not just another artifact in your client?

curveball · Bandwidth vs the connection-pool bug

Because I separated the two with evidence, and the artifact had a different signature. The early TTFT inflation — 746ms vs the oracle's 159ms — was a client-side bug: my httpx connection pool was sized 80 but peak in-flight was 141, so requests queued in my own client. I ruled out coordinated omission first by recording intended-vs-actual send times and seeing schedule_delay was only ~1.5ms, then I found and fixed the pool, and every metric converged to vllm bench serve within ~1-6 percent. Crucially, the bug pointed at queueing; the corrected data pointed at decode/memory-bandwidth — and that conclusion is anchored in GPU telemetry, not the client. The signature is unambiguous: util ~89 percent but power plateaued ~350W below TDP, and KV cache never filled at 76 percent. A client artifact can't produce 'compute units idle, power below limit' on the GPU — that's the hardware stalling on HBM. So the proof is two independent witnesses agreeing: the corrected client metrics cross-validated against the official oracle, and the nvidia-smi telemetry showing the bandwidth-stall fingerprint.

> Follow-ups: What single piece of telemetry, if it had been different, would have falsified the bandwidth-bound conclusion? · How did the pool bug deflate TPOT while inflating TTFT — and why does that detail matter for trusting the fix?

Measurement & statistics

Your harness can drive load two ways, open-loop and closed-loop. Walk me through the difference, and why you went to the trouble of supporting both instead of just one.

warmup · Open-loop vs closed-loop load generation

Open-loop fixes the arrival rate: requests show up at a target QPS regardless of whether the server is keeping up. Closed-loop fixes concurrency: you have N workers and each one only fires its next request after the previous response comes back. The reason both matter is they answer different questions. Open-loop models real internet traffic, where arrivals are exogenous, so it's the only one that exposes overload and queueing, when offered load exceeds capacity, the backlog and tail latency blow up and you see the system actually fall over. Closed-loop never lets that happen, because if the server slows down, the clients automatically slow down too, so it measures max sustainable throughput at a fixed parallelism but it structurally can't show you a saturation cliff. In my code the open path pre-schedules absolute send deadlines from a Poisson schedule and fires each request with create_task without awaiting the previous response, so a slow server can't throttle the offered load. The closed path is a fixed worker pool pulling off a queue. I led my headline result, raw ceiling about 16 req/s, useful capacity under SLO about 7, with the open-loop sweep precisely because the interesting story is where and how goodput collapses, and only open-loop shows that.

> Follow-ups: In closed-loop, what does TTFT even mean relative to open-loop? · If closed-loop can't show overload, when would you actually prefer it? · What QPS would a closed-loop run with concurrency 8 settle at near saturation?

Coordinated omission is the classic load-testing trap. Explain what it is, and then show me concretely where in your design it would have bitten you and how you defend against it.

core · Coordinated omission

Coordinated omission is when the load generator slows its own sending in lockstep with the server slowing down, so the worst-affected requests never get sent and the tail just disappears from your data. The textbook example is a closed-loop client: if every worker waits for its response before sending the next one, then when the server stalls, you simply stop generating the requests that would have suffered the most, so your P99 looks great precisely when the system is at its worst. My defense in the open path is two things working together. First, I decide every request's send time in advance from a Poisson schedule, anchored to absolute deadlines, and I fire on schedule with create_task without awaiting the prior response, so a slow server can never gate my offered load. Second, and this is the part that makes it auditable, every record stores both the intended send time and the actual send time, and the headline TTFT and E2E are computed relative to the intended arrival, not the actual POST. That corrected anchor is the honest, user-felt latency: if a request was supposed to go out at t=5s but my client only managed to issue it at t=5.2s, that 200ms of client backlog is charged to the latency, not hidden. Under open-loop saturation the corrected number reads higher than raw service time, and that gap is exactly what a naive harness omits.

> Follow-ups: You said you record intended vs actual send time. What did schedule_delay actually measure in your run, and what did that rule out? · For the closed loop you said corrected equals service by construction. Why is that, and is that a problem? · Does firing with create_task fully solve coordinated omission, or just move it somewhere?

Your TTFT once came out 4.7x too high, 746ms versus the oracle's 159ms. Take me through how you diagnosed that, given that an inflated TTFT looks exactly like server-side queueing.

hard · The connection-pool bug and disambiguating client vs server delay

That's the heart of the project. An inflated TTFT is genuinely ambiguous: it could be the server queueing requests it can't admit, or it could be my own client holding requests back before they ever hit the wire. The discriminator is the intended-vs-actual instrumentation I'd already baked in. I looked at schedule_delay, actual_send_ts minus intended_send_ts, and it was only about 1.5ms. That immediately ruled out coordinated omission and client-side scheduling lag as the cause, because if my event loop were falling behind on firing tasks, schedule_delay would be large. So the request was being issued on time, but first-token was still late. The actual culprit was one layer down: httpx's connection pool was sized at 80, but under open-loop saturation the emergent in-flight count spiked to 141, far above the nominal rate because each request now takes multiple seconds. So requests were issued on schedule into httpx, then queued inside the client waiting for a free connection, before the POST ever went out. That queueing inflated TTFT and, because the tokens then arrived in a burst once the connection freed, it actually deflated TPOT. Sizing the pool generously, for open-loop I now hard-code 1024 connections, made the client a non-bottleneck and every metric converged to the oracle within roughly 1 to 6 percent. The deeper lesson: the buggy data had pointed at queueing as the diagnosis, and the corrected data pointed at memory-bandwidth-bound decode, which is what the GPU telemetry independently said all along.

> Follow-ups: Why does a too-small client pool deflate TPOT specifically, not just inflate TTFT? · You now stamp actual_send_ts before acquiring the client semaphore. Why does the order matter? · How would you catch this class of bug automatically in CI, not by eyeballing the oracle?

You report P95 and P99, not just means. Make the case for that, and then tell me the failure mode of reporting a P99, because there's a subtle one your code guards against.

core · Percentiles vs averages and minimum sample size

Averages are reported because they're easy, but they're the wrong number for serving because the mean hides the unlucky requests and the tail is what breaks SLAs. A request that's 10x slower than median is

invisible in an average that's dominated by the bulk, but it's exactly the request that times out a user. So the headline is P95/P99 on TTFT, TPOT, and E2E, where the SLA actually lives. The subtle failure mode is that a percentile is only meaningful if you have enough samples to estimate it. A P99 computed from 30 samples is fabricated, the 99th percentile of 30 points is basically just the max with a fancy name, and it'll swing wildly run to run. So my percentile function has a minimum-sample-size guard: it returns NaN rather than a number when the sample count is below a threshold tied to the quantile, 100 samples for P99, 20 for P95, 2 for the median. Returning NaN is a deliberate, documented choice; I'd rather show a hole in the table than print a confident-looking P99 that's noise. That's actually a known divergence from the vLLM oracle, which always interpolates regardless of sample count, so I had to account for it when comparing. I also never average pre-computed percentiles when merging runs, a P99 of P99s is not a P99, I re-pool the raw per-request samples and recompute once.

> Follow-ups: Why 100 for P99 specifically? Where does that number come from? · If the oracle always interpolates and you NaN out, how did you keep the cross-check fair? · What interpolation method do you use, and does the choice matter at P99?

Throughput sounds trivial, requests over time, but there are at least two wrong ways to compute the denominator. How exactly do you define your throughput window, and why that way?

hard · Window-based throughput definition

The tempting but wrong way is last-request-finish minus first-request-start, derived from the request timestamps themselves. That's wrong under open-loop because at the start of a cell the pipeline is filling and at the end it's draining, so deriving the window from request timestamps either double-counts ramp time or, worse, shrinks the denominator and inflates throughput. What I actually use is a single perf_counter delta measured by the orchestrator around the send-plus-gather of the measurement window, this mirrors vLLM's benchmark_duration. The numerator is the count of successful requests, or output tokens for token throughput, over that fixed wall-clock window. Three details make it honest: the window is the harness-level timer, not the request span; failures are excluded from the numerator so a fast error doesn't pad throughput; and warmup requests are dropped before the window opens, so cold-start and JIT effects don't pollute the steady-state number. The reason this matters for my result is that achieved throughput plateaued at about 16 req/s even as offered load climbed to 32, and you only see that plateau cleanly if the denominator is a fixed window. A timestamp-derived window would have hidden the saturation by quietly stretching or shrinking with the load.

> Follow-ups: If a request is still in flight when the window closes, how do you count it? · Offered QPS was 32 but achieved was ~16. Where do the other 16 req/s of work go in your accounting? · Why is window_dur_s pulled from meta rather than recomputed in the reporter?

Your raw ceiling was 16 req/s but you report useful capacity as about 7. That number comes from goodput. Define goodput precisely as you compute it, including the SLO, and tell me why it's the number you actually optimize.

core · Goodput and SLO definition

Goodput is throughput counting only the requests that met the SLO; everything else is work the GPU did that nobody can use, because a token delivered too late is worthless. My SLO is a strict conjunction: a request is good only if it succeeded AND TTFT <= 1000ms AND TPOT <= 50ms. Both bounds, not either. I compute it per cell as the count of good requests over the same fixed window I use for throughput, so goodput is in req/s directly comparable to raw throughput, and I also emit an attainment fraction, good over successful, which is a DistServe-style SLO-attainment number. A couple of correctness details: the comparison is non-strict, value <= bound passes, with a one-nanosecond epsilon so floating-point

timestamp math times 1000 doesn't knock a request that's exactly on the line out of the good set; and a single-token output has undefined TPOT, so per vLLM's convention I treat its TPOT as 0, which trivially satisfies any TPOT bound. The reason this is the number I optimize rather than raw throughput: my goodput peaks around 7 req/s and then collapses to 0 by rate 32, even though raw throughput is still around 16. Past the knee you're producing tokens that all violate the SLO, so raw throughput is lying to you about capacity. The optimal serving config sits just below the knee, where goodput is maximized.

> Follow-ups: Which of the two bounds, TTFT or TPOT, breaks first as you ramp load, and what does that tell you about the bottleneck? · Why conjunction and not, say, only gating on E2E latency? · If a customer only cared about TTFT, how would your reported capacity change?

Errors are where benchmarks quietly cheat. Walk me through your failure taxonomy and the exact rule for whether a request's latency enters your statistics.

core · Failure handling and what counts as a sample

The cardinal rule is that failed requests never enter the latency or throughput sample arrays, they're tracked separately in a per-class error table, because a fast error is the most dangerous data point there is: if a timeout or a 500 returns in 5ms and you let it into your latency array, it pulls your mean and even your percentiles down and makes an overloaded server look fast. So latency and throughput are computed over successful requests only, and failures are counted by class. My taxonomy isn't just success/fail, it has distinct statuses: HTTP_ERROR with the status code, TIMEOUT, CONNECTION_ERROR, TRUNCATED_STREAM for a stream that ended without [DONE], MISSING_USAGE for a stream that finished but never sent a usage chunk, and EMPTY_OUTPUT. The reason MISSING_USAGE is its own failure and not a quiet success is important: a stream that ends with zero tokens and no usage would otherwise sneak into the good set as a 0-token, 0-TPOT request and trivially pass every SLO, which would fabricate goodput. So I explicitly demote it to a failure. I also guard the success path itself: if a record is coded SUCCESS but is missing a first or last token timestamp, finalize re-classifies it as a failure rather than letting None leak into a percentile computation. And in the percentile and summary functions I filter None and NaN out of the sample arrays defensively, so even a malformed record can't poison an aggregate.

> Follow-ups: Success rate dropped as you ramped. How do you keep a falling success rate from making latency look artificially better? · Why is a truncated stream a failure rather than just a short success? · Single-token successes get TPOT=0 and pass the SLO. Isn't that the same loophole you just closed for empty outputs?

Anyone can produce numbers. How do you know yours are trustworthy? Be specific about what the cross-check covers and what it can't.

hard · Trustworthiness via the oracle cross-check

My trust argument has three legs. The first and strongest is differential testing against vLLM's own vllm bench serve as a reference oracle. I run the same sweep point through my custom load generator and through the official tool, and they agree to within about 1 to 6 percent on throughput, TPOT, and E2E. The discipline that makes this honest is that both the custom data and the oracle data are aggregated by the exact same code, my metrics module is the single ruler, so I'm comparing measurement methodologies, not two different definitions of P99. I also matched the oracle's choices deliberately where it mattered: same arrival law, gamma with shape equal to burstiness, which reduces to exponential Poisson at burstiness 1; the exact vLLM TPOT formula, decode span over output_len minus 1; and token-weighted pooled ITL. The second leg is the value of the cross-check, it's not a rubber stamp, it actually caught my connection-pool bug, the 4.7x TTFT error, which is the whole reason I trust it: a check that has never

failed hasn't been tested. The third leg is physical corroboration: the corrected numbers agreed with independent GPU telemetry, util ~89%, power plateaued at 350W below TDP, KV cache only 76% full, which is the signature of memory-bandwidth-bound decode, the same story the latency data told once the bug was fixed. What the cross-check can't catch is a systematic error the oracle shares with me, if vLLM's tool and mine both define a metric wrong the same way, they'd agree and both be wrong; that's why I also keep raw JSONL so any metric is recomputable, and I anchor against the physical telemetry as a second, independent witness.

> Follow-ups: You deliberately diverge from the oracle on the min-sample NaN guard. Walk me through reconciling that in the comparison. · If you and the oracle agreed but both disagreed with the telemetry, which would you trust and why? · What's the single unit bug you most feared, and what's the one place in the code that prevents it?

Debugging & engineering judgment

Walk me through how you even noticed something was wrong. Your harness produced plausible numbers — what made you distrust them?

warmup · Discovery: how the bug surfaced at all

I didn't catch it by staring at my own numbers — they looked totally reasonable on their own: TTFT climbing under load is exactly what you'd expect, so nothing internally screamed 'bug'. I caught it because I cross-checked against an independent reference, vLLM's official 'vllm bench serve', pointed at the same server with matched params — same model, 512-in/128-out, same request rate, ignore_eos, random-range-ratio 0 so the lengths are fixed. When I lined the two up, throughput, TPOT, and end-to-end latency agreed within roughly 1 to 6 percent, but my TTFT was about 4.7x higher than the oracle's — 746ms versus 159ms. One metric being wildly off while every other metric matches is a very specific signature. If the server were genuinely slow on first token, TPOT and E2E would have moved too. So the disagreement pattern itself told me the problem was almost certainly in how I measured or generated load for TTFT specifically, not in the server. The meta-point: a self-built ruler can be confidently wrong, and the only thing that reliably catches that is an independent reference. Wrong numbers are worse than no numbers.

> Follow-ups: Why is 'one metric off, the rest matching' more diagnostic than 'everything slightly off'? · If you didn't have an oracle, what internal invariant could have flagged this? · Could the 1-6% residual on the other metrics also be a bug? How would you decide it's just noise?

Inflated TTFT under load is the textbook symptom of coordinated omission. How did you rule that out before chasing anything else?

core · Ruling out coordinated omission

Coordinated omission and a client-side stall after sending look identical on a latency chart, so you can't tell them apart by looking at the latency — you have to instrument the send path. My load generator pre-schedules every request's absolute send deadline from a Poisson process and records two timestamps: intended_send_ts (the deadline) and actual_send_ts (when I actually issued the POST). The gap, schedule_delay = actual minus intended, is exactly the coordinated-omission signal: if the generator falls behind and sends late, that gap blows up. In my data it was about 1.5ms. So I was sending on time — the offered load was honest, arrivals weren't being throttled by slow responses. That rules out coordinated omission by construction. It's a positive test, not hand-waving: coordinated omission requires late sends, schedule_delay measures late sends, and it was ~1.5ms. Importantly, the architecture is what made this

checkable — I fire each request with create_task without awaiting the previous response, so a slow server physically can't throttle my sending rate, and I stamp actual_send_ts before any client-side semaphore so an admission wait can't masquerade as on-time sending. The latency was real and felt by the 'user', but it was being injected after I'd sent, on my side, not by me failing to send.

> Follow-ups: Where exactly do you stamp actual_send_ts relative to the connection-pool acquire, and why does that ordering matter here? · If schedule_delay HAD been large, would that have proven coordinated omission, or could a too-small pool also delay the actual send? · Your TTFT uses the corrected anchor (relative to intended send). Doesn't that make TTFT look worse? Why is that the honest choice?

Once you'd excluded coordinated omission, how did you land on the connection pool specifically, and what was the actual numerical mismatch?

core · Root cause: connection pool vs in-flight

Open-loop concurrency is emergent, not something you set. You fix the arrival rate, but the number of in-flight requests is rate times latency — Little's Law. At rate 16 with multi-second end-to-end latency under load, in-flight peaked around 141 concurrent. My httpx client's connection pool, though, had been sized off the closed-loop concurrency default, around 80 connections. So the pool was the binding constraint: at any moment ~80 requests could be on the wire and the other ~60 sat in httpx's internal queue waiting for a free connection — queued inside my own client, before the bytes ever reached vLLM. That client-side wait lands entirely in TTFT, because TTFT is measured from intended send to first token and the connection-acquire wait happens in that interval. That's the whole 4.7x. The fix was just to size the pool to the real peak: for open-loop and max-throughput modes I set a generous 1024-connection pool so the client is never the bottleneck, and I raise the file-descriptor soft limit because that many sockets needs the FDs. After that, TTFT converged to the oracle's ~159ms. The general principle: in an open-loop test the client must be provisioned for peak emergent in-flight, which is far above the nominal rate — sizing the pool to the rate, or to a closed-loop concurrency, silently turns your load generator into a second, hidden server with its own queue.

> Follow-ups: Derive the ~141 from Little's Law — what latency does that imply at rate 16? · Why 1024 specifically? Isn't an oversized pool also a way to lie — couldn't you now overwhelm the server differently? · httpx's default max_connections is 100. Walk me through how a request actually blocks when the pool is exhausted.

The pool being too small inflating TTFT I follow. But you said it also deflated TPOT — that's counterintuitive. The same bottleneck made one metric too high and another too low. Explain the mechanism.

hard · Mechanism: why TPOT deflated

Right, and that asymmetry is actually a tell. TPOT is computed as the decode span divided by tokens minus one — (last_token_ts minus first_token_ts) over (output_tokens minus 1). It only measures the interval between the first and last token; it doesn't include anything before the first token. When a request is stuck waiting for a connection, that delay sits entirely before first token, so it inflates TTFT. But here's the buffering effect: while my client was blocked, vLLM had already been generating tokens for that request server-side, and those SSE chunks were buffering in the socket and the kernel. The moment my client finally got scheduled and started reading the stream, several already-produced tokens arrived back-to-back in a burst, because I was draining a backlog rather than reading at the true generation cadence. That compresses the apparent inter-token gaps, so the measured decode span is shorter than the real one and TPOT reads artificially low. So one bug, two opposite-direction errors, split exactly at the first-token boundary: pre-first-token delay inflated TTFT, post-first-token burst-draining deflated TPOT. That

split is itself evidence the problem is client-side I/O scheduling, not the model — a genuinely slow decode would push TPOT up, not down. Once the pool was big enough that I read each stream promptly, the inter-token timestamps reflected real generation cadence and TPOT matched the oracle.

> Follow-ups: If TPOT was deflated, why did the SLO still trip on TPOT first in the corrected data? · You also report ITL pooled across requests. Would ITL show the same burst artifact, and would it show it differently from per-request TPOT? · Could you have detected the buffering directly from the token_timestamps without the oracle? What would the burst look like in the raw gaps?

Here's what worries me as a reviewer: before you found the bug, the inflated TTFT was telling a story — 'the server is queueing, it's a queue/admission problem.' You believed that. How do you avoid shipping a confident-but-wrong root cause next time?

hard · Engineering judgment: the wrong diagnosis

That's the most important lesson from the whole project, and you've put your finger on it. The buggy data didn't just give a wrong number, it gave a wrong narrative: high, rising TTFT looks exactly like server-side queueing, so the obvious conclusion was 'KV cache is filling, requests are waiting, it's a capacity wall.' I almost shipped that. Two things saved it. First, the cross-check: an independent reference disagreed on precisely the metric my story depended on, which forced me to actually instrument instead of pattern-match. Second — and this is the judgment part — the GPU telemetry had been quietly contradicting the queueing story the whole time. If it were a KV-cache capacity wall, I'd expect KV occupancy near 100% and a growing wait queue. Instead KV was only ~76%, never full, util was ~89% but power plateaued around 350W, below the card's limit. That fingerprint isn't 'out of memory' or 'compute saturated' — it's memory-bandwidth-bound decode. So I had two independent sources, the oracle and the hardware counters, both pointing away from 'queue.' The corrected data then matched the telemetry: TPOT, not TTFT, was what crossed the SLO. The takeaway I carry: when a latency story and the hardware telemetry disagree, distrust the story — latency can be corrupted by your own client, but the power and bandwidth counters can't be faked by a connection pool. Triangulate across independent signals and treat the plausible-looking result as the dangerous one.

> Follow-ups: Concretely, what KV-cache and wait-queue values would have CONFIRMED the queueing story instead? · If util is 89% but it's bandwidth-bound, why isn't util a lie here too — you've said GPU-util is misleading? · Suppose the oracle had AGREED with your buggy TTFT. Would the telemetry alone have been enough to overturn the queueing diagnosis?

Step back from this one bug. Give me your general method for diagnosing what's bottlenecking an inference server purely from telemetry plus latency — the decision procedure, not the anecdote.

core · Telemetry-driven bottleneck diagnosis (general method)

I treat it as ruling out a small set of bottleneck classes with specific, falsifiable fingerprints, correlating three signal families on the same time axis: latency split into TTFT vs TPOT, GPU hardware counters (util, power-vs-limit, HBM used), and vLLM's own /metrics (KV-cache occupancy, num_running, num_waiting). The decision tree: First, which latency component is degrading? If TTFT degrades, suspect prefill (compute) or queue/admission wait. If TPOT degrades, suspect decode, which is memory-bandwidth-bound. Second, check power versus the card's limit. Prefill is compute-bound and pulls power near TDP; decode sits below TDP because each token streams all ~16GB of bf16 weights from HBM with about one FLOP per byte, so the compute units stall on memory and power can't reach the limit. Power plateauing below limit with high util is the bandwidth-bound signature. Third, check KV-cache occupancy and the wait queue: if KV approaches 100% and num_waiting grows, it's a memory-capacity wall capping concurrency — requests are genuinely queueing server-side. If KV is well under full, it is NOT a capacity wall,

regardless of what the latency looks like. In my run: TPOT crossed the SLO first, power plateaued ~350W below limit, KV only ~76% — that's decode/bandwidth-bound, and the fix is a higher-bandwidth GPU, fp8/int4 to move less data per token, or more replicas, not more compute and not more VRAM. And the load-gen lesson sits on top: before trusting any of this, confirm the client isn't the bottleneck via schedule_delay and in-flight-vs-pool, because a client artifact can forge a server-side story. One more guardrail — I never trust GPU-util alone; it only means a kernel ran during the sample, so I always pair it with power-vs-limit and the bandwidth or KV counters.

> Follow-ups: Distinguish a KV-capacity wall from bandwidth-bound decode using only the /metrics gauges — what diverges? · You separate VRAM capacity from bandwidth. Give a case where capacity is fine but bandwidth is the wall, and the opposite. · How does prompt length vs output length in your sweep let you isolate prefill-bound from decode-bound regimes?

You mentioned an unpinned dependency silently broke a run. Tell me what happened and how your reproducibility setup is built so that a benchmark result actually means something.

core · Reproducibility and version pinning

The engine binary is part of the measurement — a different vLLM version is a different scheduler, different batching, different kernels, so the number isn't comparable. I pin vLLM to 0.11.0 from a single source of truth, DEFAULT_VLLM_VERSION in config.py, and everything derives from it: the ServeConfig, the docker-compose image tag, and the requirements-gpu pin all reference that one constant so they can't drift apart. The break that taught me to be strict: transformers ships alongside vLLM and I'd left it unpinned, so pip pulled a version newer than what vLLM 0.11.0 expected, and the whole run crashed on import. That's the canonical reproducibility lesson — a transitive dependency you didn't think about can silently invalidate or kill a run — so transformers is pinned to a compatible version too. Beyond pins, every run writes an environment manifest: date, hostname, Python version, platform, and crucially nvidia-smi's GPU name, driver version, memory total and power limit, plus the actual imported torch/CUDA/vLLM versions read from the live process. So a result is never a bare number — it's a number plus the exact hardware and software stack that produced it. The principle: a benchmark you can't reproduce is an anecdote. And it ties back to the bug — the manifest also pins the load-gen config including the pool sizing, so the corrected TTFT is reproducible and you can prove the fix rather than just claim it.

> Follow-ups: Why pin the GPU driver and card model in the manifest — the model and vLLM version are the same, isn't the silicon enough to note? · Single source of truth for the version: what concretely breaks if the docker tag and the pip pin drift apart by one patch release? · Seeds: you seed arrivals and pass seed=0 to the oracle. What does seeding buy you, and what does it NOT make reproducible across GPUs?

After you bumped the pool to 1024, TTFT converged. But convergence to the oracle could be a coincidence or overfitting to one rate. How did you convince yourself the fix was correct and not just tuned to make one number match?

hard · Validating the fix / falsification

Fair challenge — making one number match by turning a knob is exactly the trap. A few things make me confident it's a real fix and not a fit. First, the fix is mechanistic, not empirical: I didn't sweep pool sizes until TTFT matched. I identified that peak in-flight was ~141 and the pool was ~80, which is a concrete deficit with a concrete consequence, and I set the pool generously above any plausible peak so the client is structurally removed as a bottleneck, rather than tuned to a value. If the explanation is right, the cause disappears entirely, it isn't balanced against. Second, I confirmed the new schedule_delay stayed small AND that in-flight no longer exceeded the pool — the mechanism's precondition is gone. Third, the convergence is across the whole metric set and across the sweep, not one cell: TTFT, TPOT, E2E, and

throughput all sit within ~1 to 6 percent of the oracle, and the previously-deflated TPOT came UP to match, which a TTFT-only fudge would not produce. If I'd only patched a TTFT calculation, TPOT wouldn't have moved. Fourth — the strongest one — the corrected data became internally consistent with a completely independent signal: the GPU telemetry already said bandwidth-bound decode, and post-fix the latency story agreed, with TPOT crossing the SLO first. A coincidental fit wouldn't reconcile the oracle and the hardware counters simultaneously. The way I'd try to falsify it: rerun at a different rate where peak in-flight is different, and confirm TTFT still tracks the oracle without re-tuning the pool. If it only matched at rate 16, I'd have overfit.

> Follow-ups: What single run would most cleanly falsify your fix — pick the rate and what you'd watch. · An over-large pool means more concurrent sockets hitting vLLM. How do you know you're not now overdriving the server and changing what you measure? · If a future result disagreed with the oracle by 8% on just TPOT, what's your first hypothesis given everything you learned here?

Systems design & scaling

7 to 70 req/s, how scale?

warmup · Replicas

Replicate to ~10 stateless vLLM replicas behind a load balancer; least-in-flight-tokens or prefix routing, not round-robin. Risks: imbalance, router SPOF, cold start.

> Follow-ups: Why is round-robin bad?

Rate 32 goodput is 0: where does admission control go?

core · Admission

Overload turns throughput to latency. Router limiter returns 429; max_num_seqs caps concurrency. Key off queue depth and predicted TPOT, not raw rate.

> Follow-ups: Why is rate keying a trap?

Decode is bandwidth-bound, power below TDP. Pick the GPU.

core · GPU choice

Buy HBM bandwidth, not FLOPs or VRAM: decode streams 16GB weights per token, so H100 at ~2x bandwidth gives ~2x throughput. Confirm with the harness.

> Follow-ups: When does VRAM matter?

Behavioral & project narrative

Tell me about this project — what is gpubench and why did you build it?

warmup · Project overview / motivation

gpubench is a single-GPU LLM inference benchmarking harness for vLLM. The motivating question was simple but slippery: how many requests per second can one GPU actually serve a model like Llama-3.1-8B at, and at what point does latency get bad enough that the throughput number becomes meaningless? So the harness has four parts. First, it serves the model with vLLM behind an OpenAI-compatible API. Second, a custom async load generator drives it open-loop — fixed Poisson arrival rates, swept from 2 to 32 req/s. Third, it samples GPU telemetry with nvidia-smi in parallel — utilization, power, KV-cache usage.

Fourth, and this is the part I care most about, every number it produces is cross-checked against vLLM's official 'vllm bench serve' tool so I know my measurements are trustworthy. On an A100 40GB the raw ceiling was about 16 req/s, roughly 2,000 output tokens/sec — but the useful capacity under an SLO of TTFT under 1 second and TPOT under 50ms peaked around 7 req/s. Past that, goodput collapses to zero by rate 32. The headline is that the honest number is less than half the raw number, and the harness is what lets me say that with confidence.

> Follow-ups: Why open-loop instead of closed-loop for the load generator? · Why did you bother cross-checking against the official tool instead of trusting your own numbers? · What's the difference between the 16 req/s ceiling and the 7 req/s useful capacity, in one sentence?

What was the hardest part of this project?

core · Hardest problem / debugging under uncertainty

The hardest part was a bug that didn't look like a bug — my early run reported TTFT about 4.7x too high, 746ms versus the oracle tool's 159ms. The hard part wasn't fixing it, it was that the wrong number told a coherent but wrong story. High TTFT looks exactly like server-side queueing, so my first instinct was that vLLM was backing up. The discipline that saved me was that I'd instrumented intended-versus-actual send times in the load generator specifically to catch coordinated omission — the classic load-tester sin where the client slows its own sending under load and hides tail latency. When I checked, the schedule_delay was only about 1.5ms, so the client was sending on time; it was NOT coordinated omission. That ruled out the obvious culprit and forced me to look at my own client. The real cause was that my httpx connection pool was sized 80 but peak in-flight requests hit 141, so requests were queueing client-side, inside my own load generator, inflating TTFT and deflating TPOT. The hard part was epistemic: I had a number that was internally consistent and pointed confidently at the wrong diagnosis. Trusting the oracle disagreement over my own plausible story is what cracked it.

> Follow-ups: How did you know it was the connection pool specifically and not something else in your client? · Why did the bug deflate TPOT while inflating TTFT — what's the mechanism? · If you hadn't had the oracle tool, how would you have caught this?

What surprised you most while doing this?

core · Surprising findings

Two things. The smaller surprise was that the GPU never looked maxed out at the wall. Utilization sat around 89%, which sounds saturated, but power plateaued near 350W — well below the card's limit — and the KV cache only reached about 76%, it never actually filled. So all three of the obvious 'we're out of capacity' signals were ambiguous. The thing that was actually saturated was memory bandwidth, which nvidia-smi doesn't give you a clean number for. That taught me decode is memory-bandwidth-bound: every output token has to stream all ~16GB of bf16 weights from HBM with only about 1 FLOP per byte of work, so the compute units stall waiting on memory and the power draw stays low because the silicon is mostly idle waiting. The bigger surprise was which latency metric broke first. I'd assumed TTFT — time to first token — would be the canary, but it stayed low until very high load. It was TPOT, the per-output-token time, that crossed the 50ms SLO first, right around rate 16. That makes sense in hindsight: prefill is a single compute-bound forward pass so it stays fast, but decode degrades as the batch grows. But it flipped my mental model of what 'overloaded' even looks like.

> Follow-ups: If util is 89% but the GPU isn't the wall, what would convince you you'd found the real bottleneck? · Why does power staying below TDP indicate a memory-bound workload? · Why does TPOT degrade with batch size while TTFT holds?

What would you do differently if you started over?

hard · Reflection / what you'd improve

Three things. First, I'd build the load generator's instrumentation — the intended-versus-actual send time recording and the connection-pool sizing — before running a single real benchmark, not after I'd already been burned. The 4.7x TTFT error cost me a wrong diagnosis; if I'd sized the pool to comfortably exceed peak in-flight from day one, or even just asserted pool_size > observed concurrency, I'd never have chased queueing. Second, I'd treat the oracle cross-check as a gate, not a final validation step. Right now 'vllm bench serve' is how I confirmed the corrected numbers converged to within 1-6%; ideally it runs alongside every sweep so any divergence trips immediately rather than after I've drawn conclusions. Third, I'd have caught the KV-cache math trap earlier — Llama-3.1-8B uses grouped-query attention with 8 key-value heads, not 32 attention heads, so per-token KV is about 128 KiB/token; if I'd naively used 32 I'd have been 4x off on memory budgeting. I got it right, but I'd bake that assumption into a test so a future model swap can't silently reintroduce the 4x error. The theme is: the things that bit me were measurement-harness bugs, not GPU mysteries, so I'd harden the harness first.

> Follow-ups: How exactly would you assert the connection pool is big enough without knowing peak concurrency in advance? · What does running the oracle 'as a gate' look like operationally? · What other silent 4x-style traps exist when you swap to a different model?

What did you learn from this project that you didn't know before?

core · Key learnings

The biggest lesson is that for LLM decode, memory bandwidth — not raw compute and not VRAM capacity — is the lever, and those last two are constantly confused. VRAM capacity in gigabytes tells you whether the model fits and how big a batch you can hold; memory bandwidth in gigabytes-per-second tells you how fast tokens actually come out. They're different specs and people conflate them. An A100 is roughly 1.5-2 TB/s, an H100 with HBM3 is around 3.3 TB/s, an H200 around 4.8 TB/s — and datacenter HBM having a very wide bus at multi-TB/s is exactly why it crushes consumer GDDR which is under 1 TB/s. That directly predicts that an H100 would roughly double my throughput, because decode is bandwidth-bound and you've roughly doubled the bandwidth — and the harness is the tool to actually confirm that rather than guess. The second lesson is about precision as a bandwidth trick: bf16 is 2 bytes per parameter so 16GB to move per token; fp8 halves the data moved so decode is roughly 2x faster, int4 roughly 4x. You're not really 'saving memory,' you're moving less data per token, which is the thing that's bottlenecked. The third, more meta lesson is to trust an independent oracle over a self-consistent story — my wrong TTFT number was internally coherent and still wrong.

> Follow-ups: Walk me through why fp8 makes decode about 2x faster but doesn't 2x the prefill. · If bandwidth is the lever, when would VRAM capacity actually become your binding constraint instead? · How would you design the experiment to confirm the H100 doubling prediction?

How would you explain what this project found to a non-expert — say, a smart product manager with no GPU background?

core · Communication / explain to a layperson

Imagine the GPU is a kitchen and each request is a dinner order. There are two stages. Reading the whole order — the prompt — is fast; the kitchen reads it all at once. That's why the time-to-first-response stays quick. But cooking the reply happens one word at a time, and for every single word the cooks have to walk to a giant pantry and haul out the entire 16-gigabyte recipe book, do a tiny bit of work, and walk back. The walk to the pantry is the slow part, not the cooking. So the kitchen isn't limited by how many

cooks it has — it's limited by how fast they can shuttle to the pantry and back. That's why I measured the GPU looking busy but not running hot: the cooks are mostly waiting in the hallway, not working. The practical punchline for a PM is this: the brochure says one GPU handles 16 orders a second, but if you actually care about customers not waiting too long, the honest number is about 7. Past that the kitchen technically still produces food, but the wait times blow past anything a user would tolerate, so the extra throughput is useless. And the single best upgrade isn't more cooks or a bigger pantry — it's a faster hallway to the pantry, which is what a newer GPU like an H100 buys you.

> Follow-ups: In your analogy, what is continuous batching — and why does it help throughput but hurt per-order time? · How would you explain to that PM why we should advertise 7 req/s and not 16? · What's the KV cache in this kitchen analogy?

Tell me about a decision you made on this project that you had to defend or that someone might push back on.

hard · Decision-making / tradeoffs and defense

The decision I'd most have to defend is reporting goodput under an SLO as the real capacity instead of the raw throughput ceiling. It's tempting to say '16 req/s' because it's the bigger, more impressive number and it's technically true — that's where output tokens-per-second tops out. But raw throughput counts requests that are arriving so late they're effectively failures from a user's perspective. So I defined capacity as goodput: throughput that actually meets the SLO, which here was TTFT under 1 second AND TPOT under 50ms, and that peaks at about 7 req/s and collapses to zero by rate 32. Someone could push back that the SLO is arbitrary — and it is a choice — but the point is that any honest capacity number has to be tied to a latency target, otherwise you're shipping a number that falls apart the moment real users hit it. The other defensible-decision was reporting P95/P99 percentiles rather than averages, with a minimum-sample-size guard so I'm not quoting a P99 off twelve data points, and excluding failed requests from the latency stats so a timeout doesn't masquerade as a fast response or pollute the tail. Averages hide exactly the tail behavior that determines whether a system feels good, so the percentile choice is the one I'd least compromise on.

> Follow-ups: If a stakeholder insists the SLO is too strict, how do you handle that conversation? · Why exclude failures from latency but still report them — wouldn't that hide problems? · Why is a minimum-sample-size guard necessary specifically for P99?

This started as a Colab experiment on one A100. How did you think about making it more than a one-off script, and what's the path to real-world use?

curveball · Engineering for reuse / platform portability

I deliberately built it to outlive the single Colab run, because a benchmark you can only run in one place isn't trustworthy. The key design decision was to make everything talk through the OpenAI-compatible API, which decoupled the load generator and telemetry from the actual serving backend. That gave me three interchangeable platforms behind one interface: a macOS mock server with no GPU at all for developing the harness logic itself, Colab with native vLLM for the real A100 measurements, and AWS or cloud where vLLM runs Dockerized via docker compose for a reproducible, isolated environment. The load generator and the analysis — the four charts: the latency-throughput knee, TTFT/TPOT versus load, GPU saturation over load and time, and goodput versus load — don't change across platforms; only the backend and the telemetry source swap, with nvidia-smi as the real-hardware backend. The path to real-world use is exactly that portability: I can develop and unit-test the harness on my Mac with no GPU cost, validate on a cheap Colab A100, then run the identical sweep on whatever production-class instance I'm actually buying — an H100 or H200 — and because the methodology is held constant and cross-checked

against the official tool, the cross-platform comparison is apples-to-apples. So the deliverable isn't the one A100 number, it's a repeatable instrument for answering 'which GPU and which precision should we actually pay for' on any model.

> Follow-ups: What does the macOS mock server actually let you test if there's no GPU? · How do you keep the methodology truly identical across Colab and Dockerized AWS? · If you ran the same harness on an H100 and didn't see ~2x, what would you suspect first?

Deeper methodology questions (the load-bearing ones)

When you compute goodput, do you check the SLO against the TTFT the server delivered, or the TTFT the user actually felt including time the request spent queued? Show me in the code and tell me why the choice changes the answer.

hard · Goodput anchor — corrected vs service-time TTFT (the load-bearing methodology choice nobody asks)

Goodput is checked against the COORDINATED-OMISSION-CORRECTED anchor, not service time. In metrics.py, compute_goodput/is_good_request default to anchor='corrected', so ttft_corrected_s = first_token_ts - intended_send_ts (measured from the request's pre-scheduled Poisson arrival deadline), NOT ttft_service_s = first_token_ts - actual_send_ts (measured from when the POST actually left). aggregate_run also defaults anchor='corrected'. This is deliberate and is the whole reason the loadgen pre-schedules absolute send times: under open-loop saturation the corrected anchor reads HIGHER than service time because it includes any backlog before the request even reached the server, which is the honest user-felt latency. For closed loop the two anchors are equal by construction (send_one sets intended_send_ts = actual_send_ts via sync_intended=True). This choice is exactly why goodput collapses to 0 by rate 32 even though service-time TTFT stays modest: corrected TTFT absorbs the open-loop queue. Note this also means goodput uses corrected TTFT but TPOT/E2E in the SLO follow the same anchor selection (is_good_request reads e2el_corrected_s too). An interviewer who knows this will ask 'goodput against which clock?' and the answer is 'the user's clock, corrected for coordinated omission.'

You have at least three different things that cap how many requests are in flight: vLLM's max_num_seqs, the KV-cache capacity, and your client's httpx pool. Distinguish all three, say which one bit you and which one would bite a real deployment, and where admission control belongs.

hard · Server-side admission cap (max_num_seqs) vs KV-cache wall vs client pool — three different concurrency limits

Three separate ceilings, easy to conflate: (1) CLIENT pool — httpx max_connections, sized at 1024 for open loop in orchestrator._client_concurrency; if smaller than peak in-flight, requests queue inside MY process (the 80-vs-141 bug). (2) SERVER admission — vLLM's --max-num-seqs (ServeConfig.max_num_seqs), the max sequences vLLM will run in one batch; beyond it, requests sit in vLLM's WAITING queue (visible as vllm:num_requests_waiting). (3) KV-cache capacity — even under max_num_seqs, if KV blocks fill (vllm:kv_cache_usage_perc -> ~100%) vLLM preempts/queues regardless. In my A100 run NONE of the server limits were the wall: KV peaked ~76% and the bottleneck was decode bandwidth (TPOT), not admission. The bug that bit me was the CLIENT pool. But for a real deployment at rate 32 where goodput is 0, admission control belongs at the SERVER edge / load balancer: reject or shed load once offered rate exceeds the goodput-maximizing point (~7 req/s here) so you don't waste GPU cycles producing tokens that miss the SLO and are useless. The harness's job is to FIND that operating point (peak goodput); admission control is what you'd deploy to PIN the system at it.

Concretely: a queue-depth or concurrency limit set so that admitted load stays left of the goodput knee, with 429s above it.

If different requests generate different numbers of tokens, your TPOT and throughput numbers are confounded by the model's mood. How did you control for that, and what would break if you didn't?

hard · Why TPOT measures hardware not the model — pinned decode length (ignore_eos + min_tokens==max_tokens)

build_completion_payload pins the decode length exactly: max_tokens == min_tokens == output_len AND ignore_eos=True, with temperature 0.0. That forces every request to emit exactly N tokens regardless of what the model 'wants' to say, so TPOT = decode_span/(N-1) and throughput = tokens/window measure the HARDWARE's steady-state decode rate, not variance in how long the model chose to talk. The oracle is matched: build_vllm_bench_serve_cmd sets --ignore-eos and --random-range-ratio 0.0 (EXACT fixed input/output lengths, not a +/- range), so the cross-check compares identical workloads. If you didn't pin it: (a) a request that hits EOS after 3 tokens has TPOT computed over a tiny, noisy decode span; (b) short outputs dominate throughput differently than long ones; (c) the cross-check would compare two different effective workloads and 'disagreement' could be workload skew, not a real bug. The single-token edge case is also handled: tpot_per_request returns None for output_tokens<=1 (decode speed undefined) and such requests are excluded from the TPOT sample set, while for goodput a single-token success counts as TPOT=0 to match vLLM's all_tpots convention.

vLLM has prefix caching that speeds up repeated prompts. Did you leave it on or off for the benchmark, and defend the choice.

hard · Prefix caching OFF as a benchmark-integrity decision

OFF, deliberately. ServeConfig.enable_prefix_caching=False and build_vllm_serve_cmd emits --no-enable-prefix-caching; the colab/aws configs set it false explicitly. Reason: the loadgen reuses one fixed prompt per (prompt_len) cell (prompt_cache in the orchestrator builds ONE exact-token prompt and every request in the cell sends it). With prefix caching on, vLLM would serve the second request onward from cached KV and report a fake-low TTFT that has nothing to do with real prefill cost — you'd be benchmarking the cache, not the GPU. The code comment is explicit: 'else repeated prompts fake low TTFT.' This is a measurement-honesty call: I want TTFT to reflect a genuine prefill of 512 tokens every time. In production you'd often WANT prefix caching (shared system prompts), so a fair caveat is that my TTFT numbers are a conservative, cache-cold floor; a real deployment with shared prefixes would see lower effective TTFT. Worth stating both halves.

You say KV-cache topped out around 76%. 76% of what? Walk me from the 40GB card to that number, because if you say 76% of 40GB you're wrong.

hard · What KV-cache '76%' actually means (occupancy of the vLLM KV budget, not of the card)

76% is occupancy of vLLM's KV-cache BUDGET, not 76% of the 40GB card. The chain: card is 40GB; I launch with --gpu-memory-utilization 0.90, so vLLM may use ~36GB. Weights are ~16GB (8.03B params x 2 bytes bf16; LLAMA31_8B.weights_gb=16.1). After ~2-3GB of activation/overhead, the remaining ~17-18GB is carved into PagedAttention KV blocks — that is the budget. vllm:kv_cache_usage_perc is the fraction of THOSE blocks in use. So '76%' means ~76% of the ~17GB KV budget was occupied at peak, i.e. roughly 13GB of KV, never the whole card. Why it matters: it proves the wall was NOT capacity. KV never hit 100%, so the server never had to start queueing for lack of KV blocks; requests weren't bottlenecked on memory CAPACITY. Combined with util ~89% but power ~350W (below the A100's ~400W limit), the fingerprint is memory-BANDWIDTH-bound decode. If someone hears '76%' and pictures 76% of 40GB (~30GB) they'd wrongly conclude it was nearly out of memory. The KV-cache per-token

math (128 KiB/token) plus the budget is what lets you convert that 76% into an actual concurrent-token count via gpubench plan / kv_capacity_estimate.

Your custom load generator and vLLM's official tool both generate arrivals. If they used different arrival processes, your cross-check would be comparing two different experiments. How did you guarantee they're the same?

hard · Arrival law identical to the oracle (gamma/burstiness) — why the cross-check is apples-to-apples

I matched vLLM's arrival law exactly. generate_arrival_schedule draws interarrival gaps from numpy gamma(shape=burstiness, scale=1/(rate*burstiness)), cumsum, then rescales to total duration n/rate — which is precisely what vllm/benchmarks/serve.py does. burstiness=1.0 makes gamma collapse to an exponential, i.e. a Poisson process (the configs set burstiness 1.0). So both tools offer statistically identical traffic: same mean rate, same Poisson interarrivals, same seed discipline. That's what makes 'our number matches the oracle within 1-6%' meaningful — a disagreement is a bug in measurement, not a difference in the experiment. If I'd used, say, uniform/deterministic spacing while the oracle used Poisson, the queueing behavior (and thus TTFT/TPOT tails) would differ legitimately and the cross-check would be worthless. Same for fixed lengths: I pin input/output token counts and the oracle uses --random-range-ratio 0.0, so neither side has length variance. The burstiness knob also lets me deliberately make traffic burstier (<1) or smoother (>1) than Poisson to stress the scheduler, but the canonical cross-check run holds it at 1.0 to match.

Define your throughput denominator precisely. You said there are wrong ways to do it — name the specific wrong one you avoided and why a single window is right.

hard · Why throughput is one perf_counter window and not last-token-minus-first-send

Numerator: successful requests (or their output tokens) only. Denominator: ONE perf_counter delta the orchestrator measures around the measurement send+gather (run_cell returns time.perf_counter()-t0), passed in as window_dur_s and used verbatim in aggregate_run (request_throughput = len(success)/w). It mirrors vLLM's benchmark_duration. The wrong ways: (1) sum of per-request rates or 1/mean-latency — meaningless under concurrency, inflates with batching. (2) last_token_ts minus first_send_ts derived from the request timestamps — this is the subtle trap: it (a) shrinks the denominator if the first request's send or last request's finish is clipped, (b) is corrupted by warmup if you don't fence it, and (c) silently changes definition between open and closed loop. By using an externally-measured wall window that brackets exactly the measurement phase (warmup fired separately, before t0), the denominator is independent of which requests succeeded or how their individual timestamps landed, so throughput can't be gamed by a single fast/slow request. The code even guards window<=0 -> NaN rather than dividing, and combine_runs re-pools raw records rather than averaging precomputed rates (averaging rates across cells is another wrong denominator).

Your headline diagnosis leans on GPU util ~89% with power below TDP. But nvidia-smi utilization is famously a liar. Explain exactly what that 89% does and doesn't mean, and what you cross-checked it against so your conclusion doesn't rest on a misleading number.

hard · GPU-util is a misleading number — why 89% doesn't mean 89% busy

nvidia-smi GPU-Util (utilization.gpu, what NvidiaSmiBackend reads into util_gpu_pct) only reports the fraction of the sampling interval during which AT LEAST ONE kernel was running. It says nothing about how much of the silicon (SMs, tensor cores) was actually doing work — you can read ~100% util while most cores idle, which is EXACTLY what memory-bandwidth-bound decode looks like: a kernel is always resident (so util is high) but it's stalled waiting on HBM (so the math units idle). So I never rest the diagnosis on util alone. The corroborating signals: (1) POWER vs limit — power sat ~350W against the A100's ~400W cap; if compute were the bottleneck, power would pin near TDP (prefill does this). Power

below TDP with high util is the bandwidth-bound fingerprint. (2) KV-cache occupancy ~76% from vLLM /metrics — rules out a capacity/admission wall. (3) Which latency metric crossed first — TPOT (decode), not TTFT (prefill/queue). classify_phase in telemetry.py encodes this heuristic and, importantly, returns 'unknown' when the discriminating DCGM occupancy/bandwidth counters are absent (NVML/smi-only), so I don't overclaim. The honest version: util is the weakest of the three signals; the conclusion stands on power-below-TDP + KV-not-full + TPOT-breaks-first, with util merely consistent. On a datacenter card I'd turn on DCGM (dram_active_frac, tensor_active_frac) to measure occupancy directly instead of inferring it.

You wrote all of this on a Mac with no NVIDIA GPU. How do you have any confidence the metric math, SSE parsing, and plotting are correct before you ever touch real hardware, and how do you make sure fake data can never masquerade as real?

hard · Testing strategy with no GPU — the mock server and how you trust metric code offline

Two pillars. (1) A vLLM-SHAPED mock server (mock_server.py) that speaks the exact OpenAI surface (/v1/completions SSE, /tokenize, /v1/models, /health) AND emits the EXACT vLLM V1 /metrics names (vllm:kv_cache_usage_perc, num_requests_running/waiting). It streams fake tokens with configurable TTFT/ITL, a deterministic jitter (hash-based, no RNG, so tests are reproducible), and a saturation model (_saturation_factor: past MAX_CONC in-flight, every token slows down) so the latency-throughput KNEE is actually visible offline. This is the offline contract the real server must match — I can exercise the entire load-gen -> metrics -> reporter -> plot pipeline end-to-end before spending a cent on a GPU. (2) Unit tests pin the load-bearing math: test_kv_cache_uses_8_kv_heads_not_32 locks the GQA 128 KiB/token (and asserts the 32-head version is exactly 4x, codifying the trap), plus tests for SSE parsing, Poisson arrivals, knee detection, and the canonical serve/oracle flags. Anti-masquerade: the SyntheticBackend sets synthetic=True on every telemetry row; summarize_window propagates it; the gpu_saturation plot stamps 'SYNTHETIC telemetry (no GPU)' in crimson; and select_backend prints a WARNING if you asked for real telemetry and silently fell back to synthetic. So fake numbers are always labeled and can never be mistaken for a measurement — which is the same trust principle as the oracle cross-check, applied to telemetry.

Your README headline says ~16 req/s ceiling, ~2000 tok/s, KV 76%. Your TEACHING.md Part II says ~13 req/s, ~1600 tok/s, KV ~32%, and a connection pool of 80 vs peak 141. Which is it? An interviewer who reads both will catch this.

hard · Internal number inconsistency across the writeups (a credibility risk in interview)

These are genuinely inconsistent across the docs and need to be reconciled before an interview, because being caught contradicting your own writeup is worse than any single number. The README/summary set (16 req/s, ~2000 output tok/s, KV peak 76%, power ~350W, util ~89%, TPOT crosses 50ms at rate 16, goodput peaks ~7) appears to be the canonical/latest figures and matches the PROJECT brief. TEACHING.md Part II carries an earlier/draft set (13 req/s, ~1600 tok/s, KV ~32%, TTFT ~160ms). The pool-bug magnitude also differs: TEACHING says pool 80 vs peak 141 (~60 queued) while the orchestrator now hardcodes a 1024 open-loop pool. Recommended fix: pick ONE canonical result table (the README's), and make TEACHING Part II and any verbal pitch quote the SAME numbers; keep the bug story's specific figures (80 vs 141, schedule_delay 1.5ms, 4.7x / 746ms vs 159ms) consistent everywhere since those are the memorable, defensible specifics. Also note the committed sample CSV (results/.../summary.csv) is a 128-in/32-out SMOKE run with rates 4/8/16/24 and no SLO breach (goodput==throughput, all attainment 1.0) — it is NOT the 512/128 headline sweep, so don't point an interviewer at that CSV as evidence of the knee.

You predict an H100 roughly doubles throughput. The A100 is ~1.5-2 TB/s and H100 HBM3 is ~3.3 TB/s. Show me the prediction is bandwidth-driven, then tell me the two failure modes where the

harness would show LESS than 2x.

hard · The H100 ~2x prediction — what the bandwidth ratio actually predicts vs what it won't

First-principles: decode is memory-bandwidth-bound — each generated token streams ~16GB of bf16 weights from HBM with ~1 FLOP/byte, so per-token decode time scales ~1/bandwidth. H100 HBM3 ~3.3 TB/s vs A100 ~1.5-2 TB/s is ~1.7-2.2x, so TPOT should drop ~2x and decode-limited token throughput should rise ~2x. That's the prediction and the harness (same code, swap the platform) is the tool to confirm it — TPOT and output_tps are the columns to watch. Where it shows LESS than 2x: (1) If the bottleneck shifts. The H100 also has far more compute; doubling decode speed can push you into a regime where prefill (compute-bound, sets TTFT) or the scheduler becomes the limiter, especially at long prompts — then throughput is gated by prefill, not bandwidth, and you won't see the full 2x. (2) Capacity vs bandwidth confusion (the trap the question hints at). An 80GB H100 has ~2x the VRAM of a 40GB A100, which lets you batch MORE concurrent sequences (bigger KV budget). That raises throughput via batching, but it's a CAPACITY win, not a bandwidth win — and if you only had a 40GB-class part the extra batching headroom vanishes while bandwidth still helps. Conversely on a memory-STARVED config you might be batch-limited, not bandwidth-limited, and the bandwidth upgrade under-delivers. So bandwidth predicts the per-token speed; capacity predicts how wide you can batch; they can each mask the other. The honest claim is '~2x on the bandwidth-bound decode metric (TPOT/output tok/s), measured, not assumed,' and I'd report TTFT separately because it won't move proportionally.

You frame fp8 and int4 as bandwidth levers, not compute levers. Tie that precisely to YOUR bottleneck, then tell me the cost — because halving the bytes doesn't halve the latency for free.

hard · Precision (fp8/int4) as a bandwidth lever — and why it's not free

Because decode is bandwidth-bound (stream all weights per token), and the data moved per token is bytes_per_param x params, cutting precision cuts the bytes moved: bf16=2 B/param (~16GB), fp8=1 B/param (~8GB, ~2x less to stream -> ~2x faster decode), int4=0.5 B/param (~4GB, ~4x). Since per-token time ~ bytes/bandwidth, halving bytes ~halves TPOT — that's why precision is a BANDWIDTH lever specifically, not a compute one (the FLOPs/token barely change). It also shrinks the weight footprint, freeing VRAM for a bigger KV budget -> more concurrent batching -> more throughput, a second-order win. The costs (why it's not free): (1) Quality — fp8 is usually near-lossless for inference, but int4 (AWQ/GPTQ) can degrade accuracy; you must validate task quality, not just speed. (2) Not a clean 2x in practice — dequantization overhead, kernel support, and the fact that KV cache and activations may stay higher precision mean realized speedup is below the byte ratio. (3) Hardware support — fp8 tensor cores need Hopper-class; on A100 fp8 gains are limited. (4) The bottleneck can move — once decode is fast enough, prefill/compute or KV bandwidth becomes the limiter. ServeConfig already exposes quantization (e.g. 'awq' noted for 16GB GPUs), so the harness can MEASURE the real speedup and the quality tradeoff rather than assume the theoretical ratio — which is the whole point of having an instrument.